线段树练习

人员

于子珈、周子一、陈洛冉、赵熙羽、牛同泽、秦显森、杨瑾硕 到课, 李子瀚 线上

上周作业检查

上周作业链接: https://cppoj.kids123code.com/contest/3841

王向东老师周日十点半C++线段树区间修改

刷新

#	用户名	姓名	编程分	时间	A	B
1	zhouziyi	周子一	200	1376	100	100
2	zhaoxiyu	赵熙羽	200	3138	100	100
3	yangjinshuo	杨谨硕	100	702	100	
4	yuzijia1	于子珈	100	901	100	
5	lizihan	李子瀚	100	902	100	
6	chenluoran	陈洛冉	100	1933	100	
7	qinxiansen	秦显森	100	1972	100	
8	niutongze	牛同泽	100	1986	100	
9	xieyakai	谢亚锴	100	2006	100	
10	siyunxin	司云心	0	0		

本周作业

https://cppoj.kids123code.com/contest/4032 (课上讲了 A ~ B 题, 课后作业是 B 题必做)

课堂表现

课上反映出来很多同学对线段树的模板还是不熟悉, 写的不熟练, 需要课下多写几遍, 线段树只要多写几遍就可以掌握熟练了。

课堂内容

[CQOI2006] 简单题

给每个区间维护一个懒标签, 代表这个区间被翻转过奇数次还是偶数次

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 1e5 + 5;
struct node {
  int l, r, add;
} tr[maxn*4];
```

```
void pushdown(int u) {
  if (tr[u].add) {
    tr[u*2].add ^= 1, tr[u*2+1].add ^= 1;
    tr[u].add = 0;
  }
}

void build(int u, int l, int r) {
  tr[u] = {l, r};
  if (l == r) return;
  int mid = (l + r) / 2;
  build(u*2, l, mid), build(u*2+1, mid+1, r);
}

void modify(int u, int l, int r) {
  if (tr[u].l>=l && tr[u].r<=r) { tr[u].add ^= 1; return; }

  pushdown(u);
  int mid = (tr[u].l + tr[u].r) / 2;
  if (l <= mid) modify(u*2, l, r);
  if (r > mid) modify(u*2+1, l, r);
}

int query(int u, int pos) {
  if (tr[u].l==pos && tr[u].r==pos) return tr[u].add;

  pushdown(u);
  int mid = (tr[u].l + tr[u].r) / 2;
  if (pos <= mid) return query(u*2, pos);
  return query(u*2+1, pos);
}

int main()
{
  int n, m; cin >> n >> m;
  build(1, 1, n);

  while (m -- ) {
    int op; cin >> op;
    if (op == 1) {
      int l, r; cin >> l >> r; modify(1, l, r);
    } else {
      int x; cin >> x;
      cout << query(1, x) << endl;
    }
  }
  return 0;
}
```

小白逛公园

维护 sum, lmax, rmax, vmax 四个信息

分别代表 区间和, 前缀最大子段, 后缀最大子段, 内部最大子段 4 个信息

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 5e5 + 5;
const int inf = 0x3f3f3f3f;
struct node {
  int l, r;
  int sum, lmax, rmax, vmax;
} tr[maxn*4];
int w[maxn];

void pushup(int u) {
  node &uu = tr[u], &ll = tr[u*2], &rr = tr[u*2+1];
  uu.sum = ll.sum + rr.sum;
  uu.lmax = max(ll.lmax, ll.sum + rr.lmax);
  uu.rmax = max(rr.rmax, rr.sum + ll.rmax);
  uu.vmax = max({ll.vmax, rr.vmax, ll.rmax+rr.lmax});
}
void build(int u, int l, int r) {
  tr[u] = {l, r, 0, 0, 0, 0};
  if (l == r) { tr[u].sum = tr[u].lmax = tr[u].rmax = tr[u].vmax = w[l]; return; }

  int mid = (l + r) / 2;
  build(u*2, l, mid), build(u*2+1, mid+1, r);
  pushup(u);
}
void modify(int u, int pos, int k) {
  if (tr[u].l == tr[u].r) {
    tr[u].sum = tr[u].lmax = tr[u].rmax = tr[u].vmax = k; return;
  }

  int mid = (tr[u].l + tr[u].r) / 2;
  if (pos <= mid) modify(u*2, pos, k);
  else modify(u*2+1, pos, k);
  pushup(u);
}
node query(int u, int l, int r) {
  if (tr[u].l>=l && tr[u].r<=r) return tr[u];

  int mid = (tr[u].l + tr[u].r) / 2;
  if (r <= mid) return query(u*2, l, r);
  if (l > mid) return query(u*2+1, l, r);

  node a = query(u*2, l, r), b = query(u*2+1, l, r);
  node c = {a.l, b.r, a.sum+b.sum, max(a.lmax,a.sum+b.lmax), \
            max(b.rmax,b.sum+a.rmax), max({a.vmax,b.vmax,a.rmax+b.lmax})};
  return c;
}
```

```
int main()
{
  int n, m; cin >> n >> m;
  for (int i = 1; i <= n; ++i) cin >> w[i];
  build(1, 1, n);

  while (m -- ) {
    int op; cin >> op;
    if (op == 1) {
      int l, r; cin >> l >> r;
      if (l > r) swap(l, r);
      cout << query(1, l, r).vmax << endl;
    } else {
      int pos, k; cin >> pos >> k;
      modify(1, pos, k);
    }
  }
  return 0;
}
```